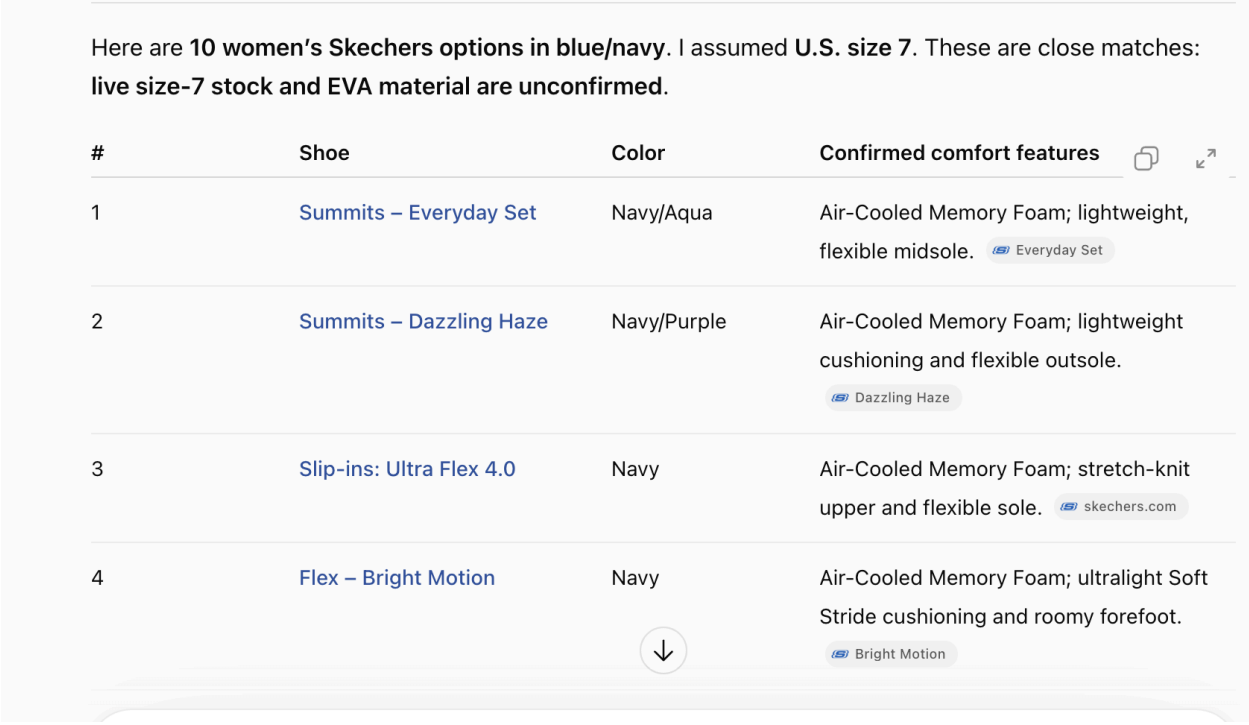
Scenario1: "Test the search feature."

ChatGBT ans without ICEPOT: Test the search feature using valid keywords, nonexistent items, empty input, spaces, special characters, and long text. Verify that results match the search, no-results messages display correctly, and submitting with either Enter or the Search button works. Check response time and confirm that selecting a result opens the correct item.

ICEPOT:
Instruction: Goto an e-commerce site, and search for Sneakers and fetch “Bata” brand.
Context: Size and color details given in this attached document[sneakers.docx].
Example: For example, In eBay site and size-7, gender-Women, blue color, Foam shoes with lightweight, cushioned footwear made from material EVA (ethylene-vinyl acetate), and memory foam.
Persona: Act as a senior QA engineer
Output: Blue color Foam footwears for women with flexibility.
Tone: Kindly list only 10 shoes.

ChatGBT Answer with ICEPOT:

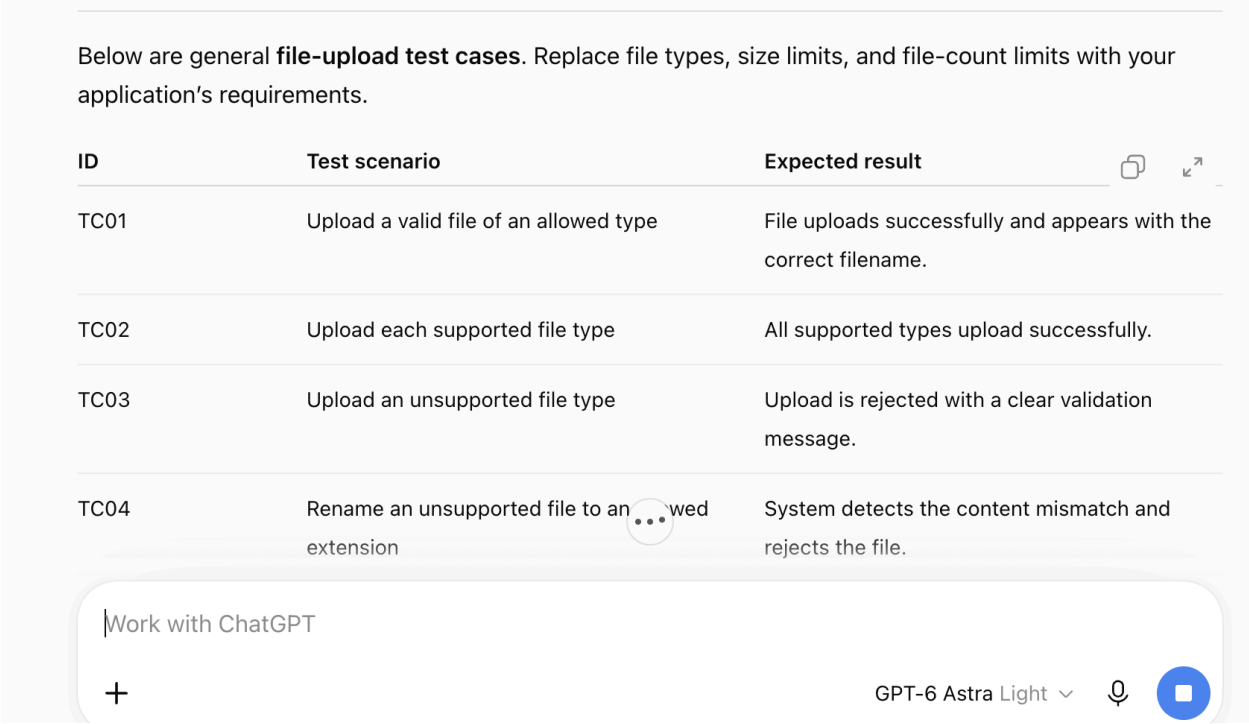


Here are **10 women’s Skechers options in blue/navy**. I assumed **U.S. size 7**. These are close matches: **live size-7 stock and EVA material are unconfirmed**.

#	Shoe	Color	Confirmed comfort features
1	Summits – Everyday Set	Navy/Aqua	Air-Cooled Memory Foam; lightweight, flexible midsole. Everyday Set
2	Summits – Dazzling Haze	Navy/Purple	Air-Cooled Memory Foam; lightweight cushioning and flexible outsole. Dazzling Haze
3	Slip-ins: Ultra Flex 4.0	Navy	Air-Cooled Memory Foam; stretch-knit upper and flexible sole. skechers.com
4	Flex – Bright Motion	Navy	Air-Cooled Memory Foam; ultralight Soft Stride cushioning and roomy forefoot. Bright Motion

Scenario2: "Generate test cases for file upload."

ChatGBT Answer without ICEPOT:



Below are general **file-upload test cases**. Replace file types, size limits, and file-count limits with your application's requirements.

ID	Test scenario	Expected result
TC01	Upload a valid file of an allowed type	File uploads successfully and appears with the correct filename.
TC02	Upload each supported file type	All supported types upload successfully.
TC03	Upload an unsupported file type	Upload is rejected with a clear validation message.
TC04	Rename an unsupported file to an …wed extension	System detects the content mismatch and rejects the file.

—----------

Scenario2: “Write API test cases for user registration."

ChatGBT ans without ICEPOT:

Below are API test cases for a user registration endpoint.

Assumption: `POST /api/register` accepts `name`, `email`, and `password`. Confirm required fields, password rules, and status codes against your API specification.

Sample valid request:

```
{
  "name": "rrb",
  "email": "rrb.test@example.com",
  "password": "rrb@123"
}
```

Test ID	Test scenario	Test data / action	Expected result

REG-01	Register with valid details	Valid name, unique email, valid password	`201 Created`; user ID returned; account created
REG-02	Missing required fields	Omit name, email, or password individually	`400` or `422`; clear field validation error; no account created
REG-03	Empty required fields	Send `""` for each required field	Request rejected with appropriate validation errors
REG-04	Null required fields	Send `null` for each required field	Request rejected; no server error
REG-05	Incorrect field types	Send number, boolean, array, or object instead of a string	Request rejected with field type errors

LISTED UNTIL 27 Testcases

ICEPOT:
Instruction: Write API test cases for User registration and registered successfully. In Request payload, verify first and last name, DOB, address, phone number and email as mandatory fields and rest as optional fields.
Context: In an e-commerce site, for Klarna first time enrollment for enabling installment option. Klarna user registration link is available in the User settings page. Using json file for request/response payloads and bearer token.
Example: For example, refer the attached test cases which you generated for the user enrollment for the first time in an e-commerce site.
Persona: Act as a Backend QA engineer.
Output: Need test cases for verifying request all mandatory fields in positive, negative and edge cases.
Tone: Be a technical professional.

ChatGBT ans with ICEPOT:

These test cases cover **first-time Klarna enrollment through an e-commerce site's User Settings page**, using JSON request/response payloads and a bearer token.

Scope: Backend API validation and successful registration. These are proposed test cases for your integration—not Amazon's internal test cases or Klarna's official API contract.

Assumptions to confirm against your API specification:

- Mandatory fields: `firstName`, `lastName`, `dob`, `address`, `phoneNumber`, and `email`.

- All other fields are optional.
- For this example, `address` is an object with required `line1`, `city`, `postalCode`, and `country`; `line2` is optional. State requirements depend on the country.
- DOB uses `YYYY-MM-DD`.
- A successful new registration returns `201 Created`. Validation failures return the documented `400` or `422` code—automated tests should assert the exact agreed code.
- **Registration success does not automatically mean approval for installments.**

1. Sample JSON request and response

Endpoint placeholder: `POST {{baseUrl}}/api/klarna/enrollments`

Headers:

Authorization: Bearer {{accessToken}}
Content-Type: application/json
Accept: application/json

Request fixture: `registration_valid_required.json`

```
{
  "firstName": "rrb",
  "lastName": "Test",
  "dob": "1990-09-21",
  "address": {
    "line1": "123 Test Street",
    "city": "Oakland",
    "postalCode": "94612",
    "country": "US"
  },
  "phoneNumber": "+12025550123",
  "email": "rrb.enrollment@example.com"
}
```

Use provider-approved sandbox phone numbers and emails when the integration sends verification messages.

Illustrative response fixture: `registration_success_expected.json`

```
{
  "registrationId": "<non-empty-generated-id>",
  "status": "REGISTERED",
  "message": "User registered successfully"
}
```

The generated ID is a placeholder: validate its type and format instead of comparing it literally.

Illustrative validation response:

```
{
  "code": "VALIDATION_ERROR",
  "errors": [
    {
      "field": "firstName",
      "code": "REQUIRED",
      "message": "First name is required"
    }
  ]
}
```

2. Positive test cases

ID	Scenario	Request / execution	Expected result
POS-01	Register with mandatory fields only	Submit the valid request fixture with a valid bearer token	`201`; registration ID returned; status is `REGISTERED`; exactly one enrollment created
POS-02	Register with all supported optional fields	Add valid optional fields defined in the schema	Registration succeeds; optional values handled according to contract
POS-03	Omit optional fields individually	Remove one optional field per request	Registration succeeds without missing-field errors
POS-04	Valid international names	Use supported Unicode names, such as `José` or `தமிழ்`	Names accepted and preserved correctly

POS-05	Names with valid punctuation	Use `Anne-Marie` and `O'Connor`	Accepted according to documented name rules
POS-06	Valid DOB	Submit a valid calendar date within the eligible age range	Registration succeeds
POS-07	Valid country-specific address	Submit address and postal code valid for a supported country	Registration succeeds
POS-08	Valid phone number	Submit a phone number in the documented format	Accepted; normalization follows contract
POS-09	Valid email variants	Use supported email formats, including a plus alias	Accepted according to email validation rules
POS-10	Verify persisted registration	Retrieve enrollment through an authorized API or test database	Stored values match the accepted request; enrollment belongs to the authenticated user

3. Mandatory-field negative tests

Run every row below separately for each mandatory field:
`firstName`, `lastName`, `dob`, `address`, `phoneNumber`, and `email`.

For address text checks, also apply them to each required address component.

ID	Scenario	Payload modification	Expected result
MAN-01	Mandatory field missing	Remove the field entirely	Validation error identifies the missing field; no enrollment created
MAN-02	Mandatory field is null	Set field to `null`	Validation error; no enrollment created

MAN-03	Mandatory field is empty	Set string field to `""`; set address to `{}`	Field or required address-component errors returned
MAN-04	Mandatory field contains whitespace only	Set string field to `"  "`	Rejected as empty/invalid after documented whitespace handling
MAN-05	Wrong JSON type	Send number, boolean, array, or object where a string is required; send a string for address	Type validation error; no `500` response
MAN-06	Incorrect field-name casing	Replace `firstName` with `FirstName`, for example	Required-field error if schema is case-sensitive; incorrectly named key does not satisfy requirement
MAN-07	Multiple mandatory fields missing	Remove first name, DOB, and email	Documented validation errors returned; no partial enrollment
MAN-08	All mandatory fields missing	Submit `{}`	Request rejected with required-field validation errors

4. Field-specific negative and edge cases

Use configured limits from the API contract instead of assuming fixed name lengths, age thresholds, or phone lengths.

ID	Field	Scenario / test data	Expected result
FLD-01	First and last name	Test each field at minimum and maximum permitted lengths	Boundary values accepted
FLD-02	First and last name	Minimum minus one and maximum plus one characters	Invalid lengths rejected

FLD-03	First and last name	Leading/trailing spaces, such as `" rrb "`	Trimmed or rejected according to contract; no unintended data corruption
FLD-04	First and last name	Digits only, unsupported symbols, or control characters	Documented validation enforced; no server error
FLD-05	DOB	Wrong format: `"09/21/1990"` or timestamp instead of date	Rejected when contract requires `YYYY-MM-DD`
FLD-06	DOB	Impossible date: `"1990-02-30"` or `"1990-13-01"`	Calendar validation error
FLD-07	DOB	Future date or today's date	Rejected under enrollment age rules
FLD-08	DOB	Exactly the minimum eligible age today	Accepted if all other requirements pass
FLD-09	DOB	One day younger than minimum eligible age	Rejected with eligibility validation error
FLD-10	DOB	One day older than minimum eligible age	Accepted if otherwise eligible
FLD-11	DOB	Valid leap date `"2000-02-29"` versus invalid `"2001-02-29"`	Valid date accepted if eligible; invalid date rejected
FLD-12	DOB	Oldest permitted DOB and one day beyond it, if a limit exists	Configured boundary enforced
FLD-13	Address	Omit `line1`, `city`, `postalCode`, or `country` individually	Required-component validation error
FLD-14	Address	Omit state for a country where state is required	Country-specific validation error

FLD-15	Address	Invalid or unsupported country code	Rejected with country validation error
FLD-16	Address	Postal code incompatible with country	Rejected if country-specific postal validation is part of the contract
FLD-17	Address	Postal code with leading zero, such as `"02108"`	Leading zero preserved; postal code treated as a string
FLD-18	Address	Component lengths at maximum and maximum plus one	Maximum accepted; over-limit rejected
FLD-19	Phone	Alphabetic characters, missing country code, or invalid prefix	Rejected according to required phone format
FLD-20	Phone	Minimum/maximum allowed digits and values outside limits	Country-aware length rules enforced
FLD-21	Phone	Spaces, parentheses, or hyphens	Normalized or rejected consistently according to contract
FLD-22	Email	Missing `@`, missing domain, multiple `@`, embedded spaces	Email validation error
FLD-23	Email	Leading/trailing whitespace and mixed case	Documented normalization applied consistently
FLD-24	Email	Maximum supported length and maximum plus one	Boundary accepted; over-limit rejected
FLD-25	Email / phone	Values already associated with another enrollment	Documented uniqueness/account-linking rules enforced; no unauthorized account association

5. Optional fields, authentication, and request handling

ID	Scenario	Request / execution	Expected result

API-01	Optional field supplied with invalid value	Send wrong type, format, or excessive length	Rejected if supplied value violates schema; optional does not mean unvalidated
API-02	Optional field is null or empty	Send `null` or `""` individually	Accepted only where schema permits
API-03	Missing bearer token	Omit `Authorization` header	`401 Unauthorized`; no enrollment created
API-04	Invalid, expired, or tampered token	Execute separately for each token condition	`401`; no enrollment created
API-05	Insufficient token permissions	Valid token without required scope	`403 Forbidden`; no enrollment created
API-06	Attempt to enroll another user	Supply another user's identifier, if the API accepts one	Unauthorized association rejected; enrollment remains bound to authorized identity
API-07	Malformed JSON	Remove comma/brace or use trailing comma	`400`; no enrollment created
API-08	Empty request body	Submit no JSON body	Request rejected with controlled error
API-09	Unsupported content type	Send XML or plain text	`415` when unsupported
API-10	Client-controlled approval fields	Add `"approved": true` or `"status": "REGISTERED"`	Rejected or ignored; client cannot set approval or registration status
API-11	Injection payloads	Submit SQL/script strings in text fields	No command/query execution; safely handled; validation follows contract
API-12	Oversized request	Exceed configured payload limit	Controlled rejection, typically `413`; service remains available

6. Successful registration and integration checks

ID	Scenario	Execution	Expected result
INT-01	Success response contract	Validate response against JSON schema	Exact status code, required keys, types, and allowed status values match contract
INT-02	Success message	Inspect successful response	Documented success message returned; assert exact wording only if contractually fixed
INT-03	Registration visible in User Settings	Read settings/enrollment status after success	Correct enrollment state exposed for the authenticated user
INT-04	Installment eligibility stays separate	Register a user without an installment approval decision	API does not incorrectly report installments approved solely because registration succeeded
INT-05	Repeat enrollment	Submit same user's enrollment again	Existing enrollment or documented conflict returned; no duplicate
INT-06	Concurrent enrollment	Submit two simultaneous requests for the same user	At most one enrollment created
INT-07	Retry after lost response	Retry after server processes request but client times out	No duplicate provider/local enrollment; documented idempotency behavior enforced
INT-08	Provider validation rejection	Simulate Klarna rejecting submitted data	Accurate mapped error; local state not incorrectly marked registered
INT-09	Provider timeout or outage	Simulate timeout or unavailable provider	Documented pending/error state; no false success; retry/reconciliation supported
INT-10	Local save fails after provider success	Simulate persistence failure	Failure reconciled without duplicate enrollment; final local/provider state consistent

INT-11	Sensitive data protection	Inspect responses and accessible application logs	No bearer token, stack trace, or unnecessary DOB/address/phone exposure
INT-12	Performance	Execute under agreed expected load	Response times and error rates meet the defined SLA

Execution guidance: Keep positive and negative request fixtures in JSON files, validate responses using JSON schemas, and inject bearer tokens at runtime. For each validation failure, verify both the field-specific error and **absence of a newly created enrollment**.

—----------

Scenario3: “Test the reports section."

ChatGBT ans without ICEPOT:

Please share the website URL or a screenshot of the Reports section. Do you want test cases written or testing performed on the application?

For backend API testing, also share the reports endpoint or API documentation and a sample request/response. Do not paste bearer tokens or passwords here.

ICEPOT:
Instruction: Generate a report to fetch total P1 tickets.
Context: WebPage: Search Page, The environment: “Production”, duration: “last 7 days”. Provide segregated Open, Resolved and In Progress tickets, created, resolved date and its Assignee.
Example: 5 tickets are P1 and 4 closed and 1 still in Open and Assignee is working with the other teams.
Persona: Act as a senior QA engineer.
Output: Give the report in pdf format.
Tone: I want it in a professional way.

ChatGBT ans with ICEPOT:

Screenshot of PDF report.

P1 Incident Report | Search Page

PRODUCTION / LAST 7 DAYS / SAMPLE DATA

Reporting window: 09 Oct 2026, 12:10 to 16 Oct 2026, 12:10 PDT (America/Los_Angeles).
Selection: Search Page tickets with priority P1, environment Production, and created timestamp within this window. Status reflects the window end.

TOTAL P1	OPEN	IN PROGRESS	RESOLVED
8	2	2	4

Executive summary: Four of eight sample P1 tickets are resolved (50%). Four remain unresolved: two Open and two In Progress. Active dependencies involve Search Engineering, SRE, and the Catalog team.

Ticket / issue	Status	Created (PDT)	Resolved (PDT)	Assignee	Update / next action
SMP-1007 | Search unavailable in one region	Open	16 Oct 2026 08:20	Not resolved	Priya Shah	Working with SRE and Search Engineering to isolate regional routing failures.
SMP-1008 | Search returns empty results for all queries	Open	16 Oct 2026 10:05	Not resolved	Daniel Lee	Catalog team investigating index publication; incident triage underway.
SMP-1005 | Search API timeouts during peak load	In Progress	14 Oct 2026 15:10	Not resolved	Meera Rao	Connection-pool fix in validation; SRE coordinating rollout.
SMP-1006 | Mobile search fails after deployment	In Progress	15 Oct 2026 09:25	Not resolved	Alex Chen	API compatibility fix under regression testing with Mobile team.
SMP-1001 | Search API returns widespread HTTP 500	Resolved	10 Oct 2026 09:15	10 Oct 2026 11:45	Arun Kumar	Faulty release rolled back; search smoke tests passed.
SMP-1002 | Product index unavailable	Resolved	11 Oct 2026 13:30	11 Oct 2026 17:00	Sofia Patel	Index restored; query results verified against catalog.
SMP-1003 | Search gateway rejects valid sessions	Resolved	12 Oct 2026 08:40	12 Oct 2026 10:20	James Wilson	Gateway configuration corrected; authenticated search validated.
SMP-1004 | Search requests fail after cache update	Resolved	13 Oct 2026 16:00	13 Oct 2026 18:10	Nina Thomas	Cache change reverted; recovery confirmed through monitoring.

QA follow-up: Confirm fixes using search availability, result accuracy, authentication, and mobile regression checks. Keep unresolved incidents assigned and record cross-team updates. Resolved is a workflow status; no separate Closed status is included.

Data disclaimer: All ticket IDs, issues, people, dates, and updates are fictional examples created for this report. No production system was queried. Counts represent tickets created in the reporting window, not the full outstanding backlog.